

Projeto 2 - Servicos em Nuvem

AWS Academy Learner Lab

CRUD de Tarefas com ECS Fargate, RDS, Lambda e API Gateway

Integrantes:

Matheus Fernandes - RA: 10435788 - Backend + RDS
Fernando Lacava - RA: 10438026 - Infraestrutura AWS + Deploy
Joao Trevisol - RA: 10277893 - Frontend + Lambda

2026/05

1. Visao Geral

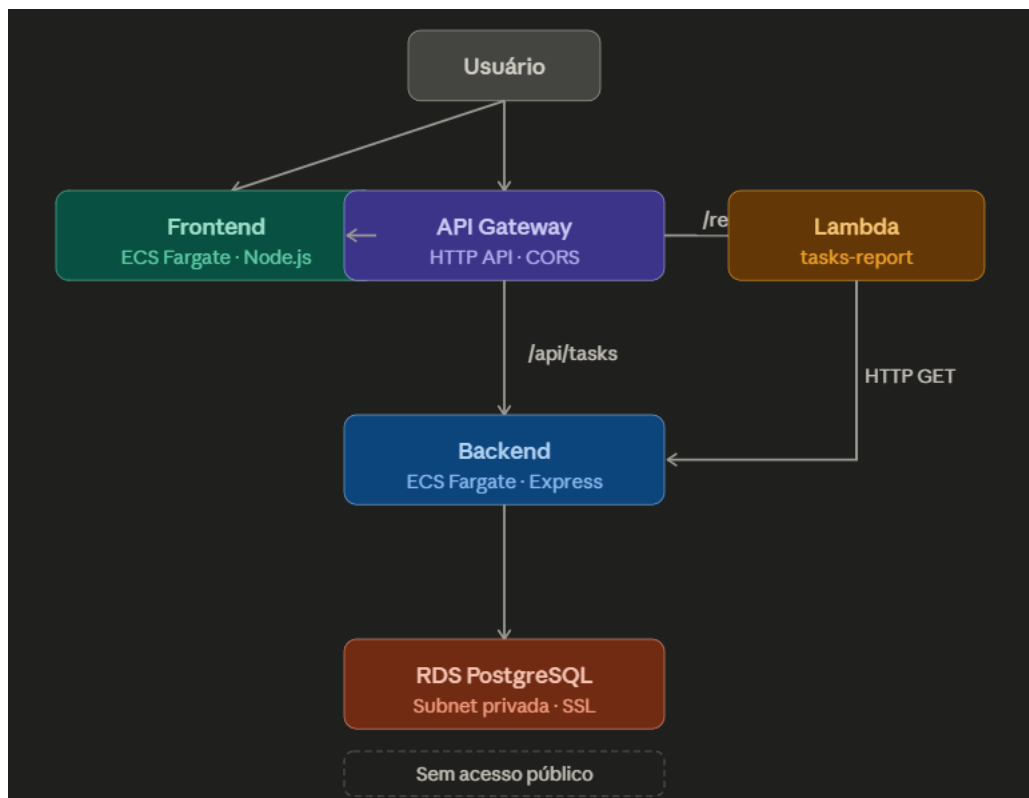
Este projeto implementa um sistema de gerenciamento de Tarefas (Tasks) utilizando servicos gerenciados da AWS. O dominio foi escolhido pela simplicidade e clareza na demonstracao dos conceitos de computacao em nuvem.

O sistema permite criar, listar, atualizar e remover tarefas com os campos: titulo, descricao, status (pendente/concluida) e prioridade (baixa/media/alta).

Responsabilidades dos Integrantes

- Matheus Fernandes (RA: 10435788): Desenvolvimento do Backend Node.js/Express, configuracao do banco RDS PostgreSQL, criacao das queries SQL e endpoints REST.
- Fernando Lacava (RA: 10438026): Configuracao da infraestrutura AWS (VPC, Security Groups, ECS Fargate, ECR), deploy dos containers e configuracao do API Gateway.
- Joao Trevisol (RA: 10277893): Desenvolvimento do Frontend em Node.js/Express, criacao da funcao Lambda /report e integracao com a API.

Diagrama de Arquitetura



2. Infraestrutura AWS

VPC e Security Groups

Foram criados 3 Security Groups na VPC default (vpc-01ee64a7bbddc4158):

Nome	Porta	Funcao
backend-sg	3000	ECS Backend - acesso publico
rds-sg	5432	RDS - apenas do backend-sg
frontend-sg	80	ECS Frontend - acesso publico

VPC Console

us-east-1.console.aws.amazon.com/vpconsole/home?region=us-east-1#SecurityGroups:

Search

Estados Unidos (Norte da Virgínia)

ID da conta: 6459-9471-7126
voclabs/user4955542-FERNANDO.LACAVA_OLIVAL...

VPC > Grupos de segurança

Painel da VPC

AWS Global View

Filtrar por VPC

Nuvem privada virtual

Suas VPCs

Sub-redes

Tabelas de rotas

Gateways da Internet

Gateways da Internet somente de saída

Gateways da operadora

Conjuntos de opções de DHCP

IPs elásticos

Listas de prefixos gerenciados

Gateways NAT

Conexões de emparelhamento

Servidores de rotas

Segurança

ACLs da rede

Grupos de segurança

Grupos de segurança (4) Informações

Encontre grupos de segurança por atributo ou etiqueta

Ações

Exportar grupos de segurança para CSV

Criar grupo de segurança

☐	Name	ID do grupo de segurança	Nome do grupo de segurança	ID da VPC	Descrição
☐	-	sg-076298c6e19988920	backend-sg	vpc-01ee64a7b8dddc4158	Backend ECS
☐	-	sg-0d7aa9b9cafb647bb	frontend-sg	vpc-01ee64a7b8dddc4158	Frontend ECS
☐	-	sg-0180616261c4ab93b	default	vpc-01ee64a7b8dddc4158	default VPC security group
☐	-	sg-01204ca545ef42489	rds-sg	vpc-01ee64a7b8dddc4158	RDS PostgreSQL

Selecione um grupo de segurança

CloudShell

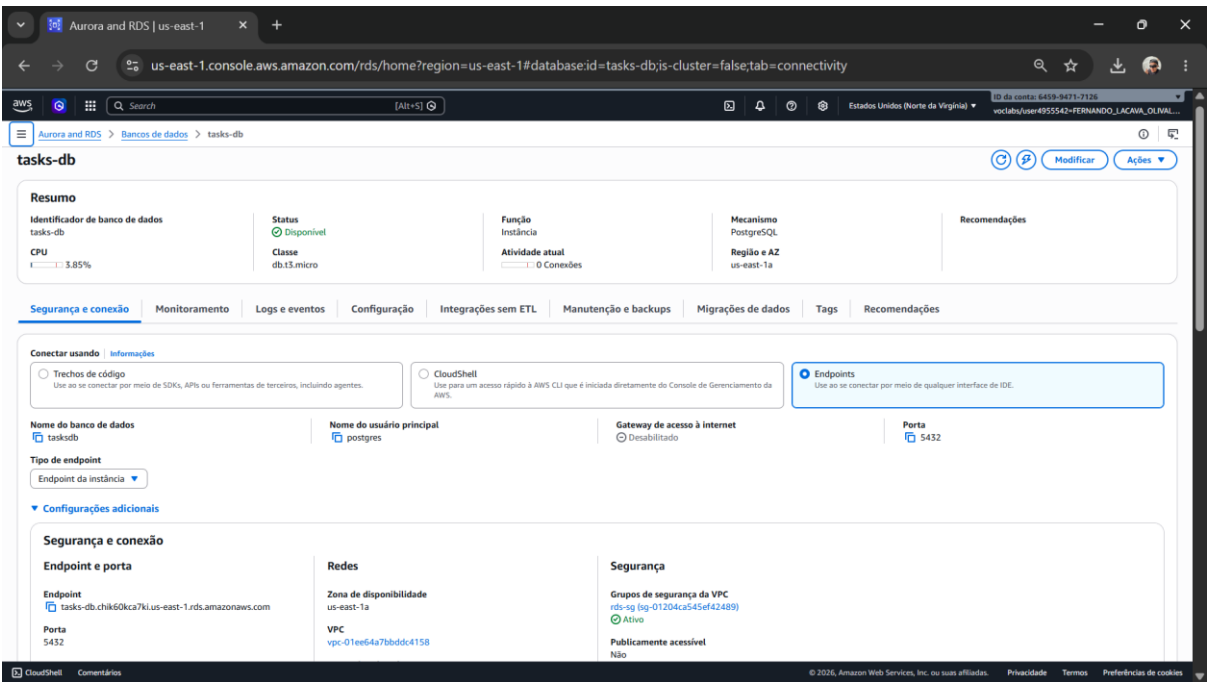
Comentários

© 2026, Amazon Web Services, Inc. ou suas afiliadas. Privacidade Termos Preferências de cookies

3. Banco de Dados RDS

O banco de dados PostgreSQL 16 foi criado no Amazon RDS com as seguintes configurações de segurança:

- Acesso publico: DESABILITADO
- Security Group: rds-sg (aceita conexões apenas do backend-sg na porta 5432)
- Instancia: db.t3.micro | Storage: 20 GB
- Endpoint: tasks-db.chik60kca7ki.us-east-1.rds.amazonaws.com
- Banco: tasksdb | Usuário: postgres



Modelo da Entidade Tasks

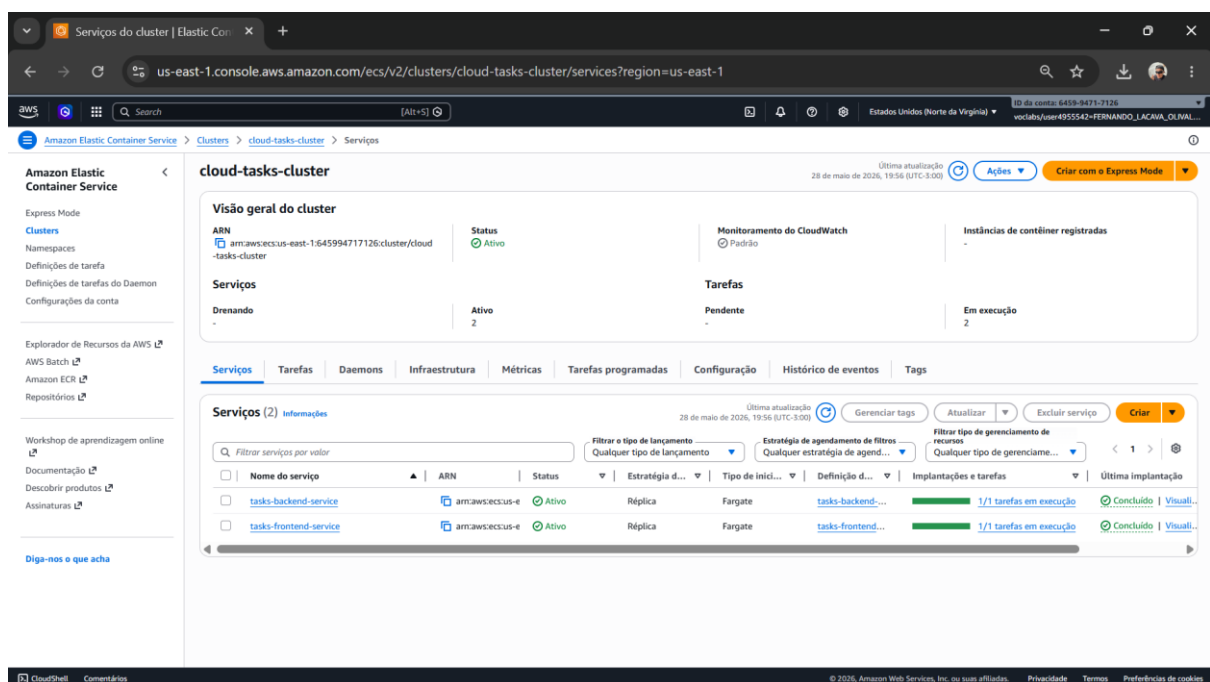
Campo	Tipo	Descrição
id	SERIAL PRIMARY KEY	Identificador unico
title	VARCHAR(100)	Titulo da tarefa
description	TEXT	Descricao detalhada
status	VARCHAR(20)	pendente ou concluida
priority	VARCHAR(10)	baixa, media ou alta
created_at	TIMESTAMP	Data de criação

4. Backend e Deploy ECS

O backend foi desenvolvido em Node.js 20 com Express, containerizado com Docker e publicado no Amazon ECR. O deploy foi realizado no Amazon ECS Fargate.

Configuracao ECS

- Cluster: cloud-tasks-cluster (Fargate)
- Task Definition: tasks-backend-task:1 (0.5 vCPU, 1 GB RAM)
- Service: tasks-backend-service (1 tarefa em execucao)
- IAM Role: LabRole (task role + execution role)
- Repositorio ECR: 645994717126.dkr.ecr.us-east-1.amazonaws.com/tasks-backend



Endpoints da API REST

Metodo	Rota	Descricao
GET	/api/tasks	Lista todas as tarefas
GET	/api/tasks/{id}	Busca tarefa por ID
POST	/api/tasks	Cria nova tarefa
PUT	/api/tasks/{id}	Atualiza tarefa
DELETE	/api/tasks/{id}	Remove tarefa

5. API Gateway

O Amazon API Gateway foi configurado como ponto central de entrada, roteando todas as requisicoes do frontend para os servicos corretos.

- Tipo: HTTP API
- Invoke URL: <https://m9aj7fdwvf.execute-api.us-east-1.amazonaws.com>
- CORS habilitado: Allow-Origin: *, Allow-Methods: *

Roteamento

Metodo	Rota	Destino
ANY	/api/tasks	ECS Backend (tasks-backend-service)
ANY	/api/tasks/{id}	ECS Backend (tasks-backend-service)
GET	/report	Lambda (tasks-report)

The image displays three sequential screenshots of the AWS API Gateway console, illustrating the configuration of routes for the 'tasks-api' API.

Top Screenshot: Shows the 'Rotas' (Routes) section for the 'tasks-api' API. The 'Rotas para tasks-api' list shows a route for '/api/tasks' with method 'ANY'. The 'Detalhes da rota' (Route details) section shows the route ID 'dwxr77', the ARN 'arn:aws:apigateway:us-east-1::apis/m9aj7fdwvf/routes/dwxr77', and the integration 'piexy46'.

Middle Screenshot: Shows the 'Rotas' section for the 'tasks-api' API. The 'Rotas para tasks-api' list shows a route for '/api/tasks/{id}' with method 'ANY'. The 'Detalhes da rota' section shows the route ID 'cw316gg', the ARN 'arn:aws:apigateway:us-east-1::apis/m9aj7fdwvf/routes/cw316gg', and the integration 'gveqgt'.

Bottom Screenshot: Shows the 'Rotas' section for the 'tasks-api' API. The 'Rotas para tasks-api' list shows a route for '/report' with method 'GET'. The 'Detalhes da rota' section shows the route ID 'rko21jm', the ARN 'arn:aws:apigateway:us-east-1::apis/m9aj7fdwvf/routes/rko21jm', and the integration '95bhnt8'.

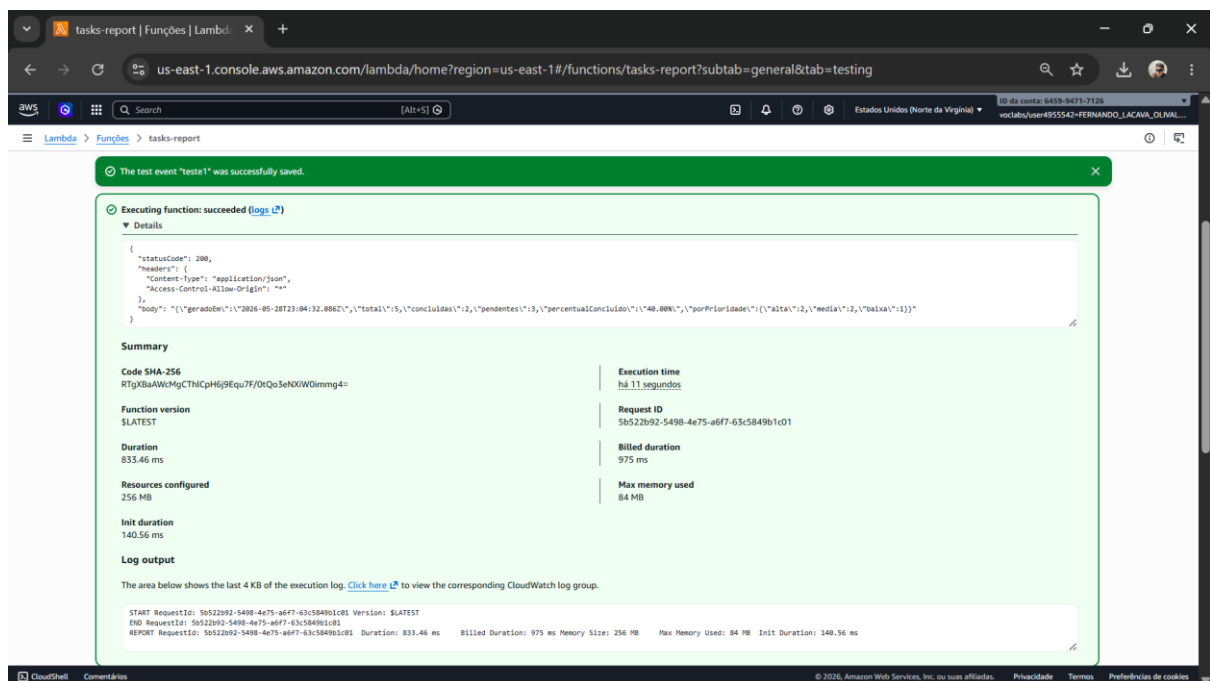
6. Funcao Lambda /report

A funcao Lambda tasks-report foi implementada em Node.js 22 e consome a API do backend via HTTP GET para gerar estatisticas das tarefas. A funcao NAO acessa o banco de dados diretamente.

- Runtime: Node.js 22.x
- IAM Role: LabRole
- Timeout: 15 segundos | Memoria: 256 MB
- Variavel de ambiente: BACKEND_URL = http://54.226.164.14:3000

Exemplo de Resposta do /report

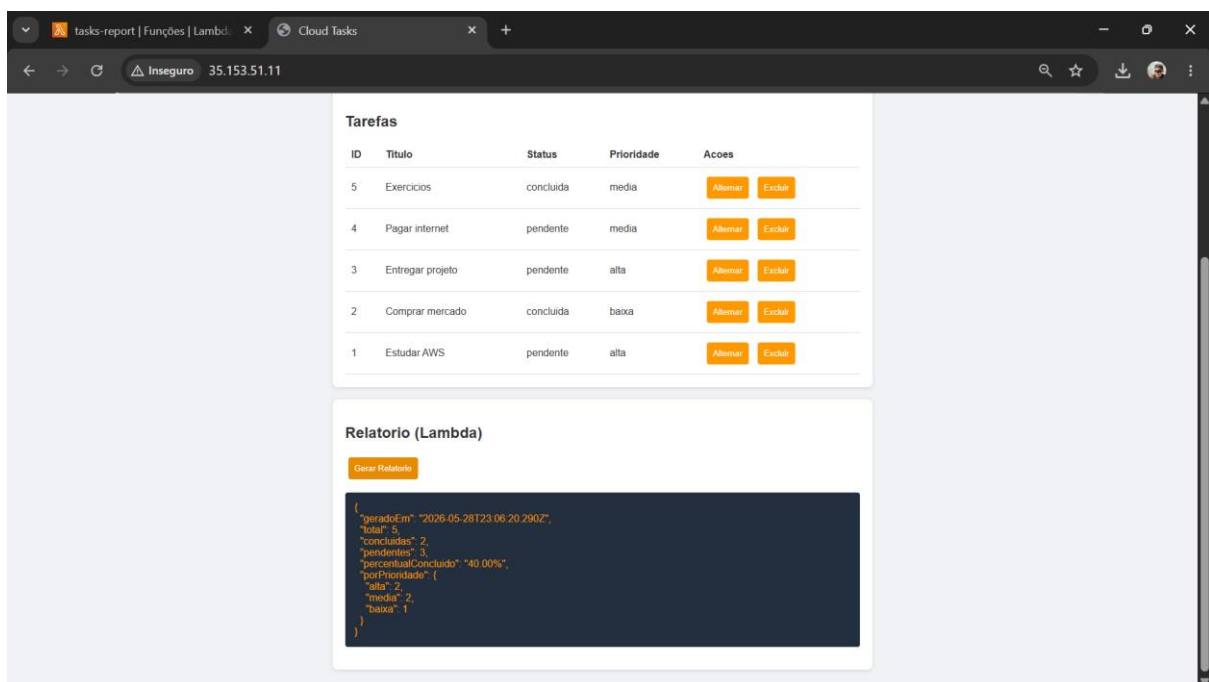
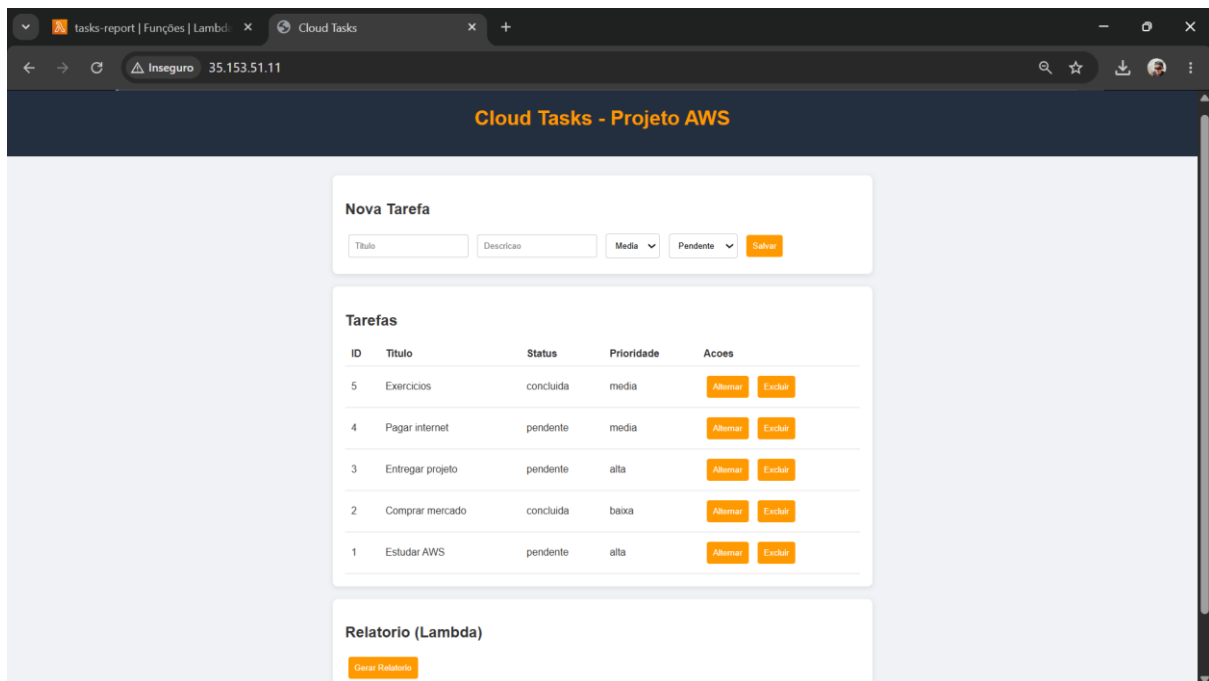
```
{ "geradoEm": "2026-05-28T22:07:29.622Z", "total": 5, "concluidas": 2, "pendentes": 3, "percentualConcluido": "40.00%", "porPrioridade": { "alta": 2, "media": 2, "baixa": 1 } }
```



7. Frontend

O frontend foi desenvolvido em HTML/CSS/JavaScript, servido por um servidor Node.js com Express (sem Nginx). O container foi publicado no ECR e executado no ECS Fargate.

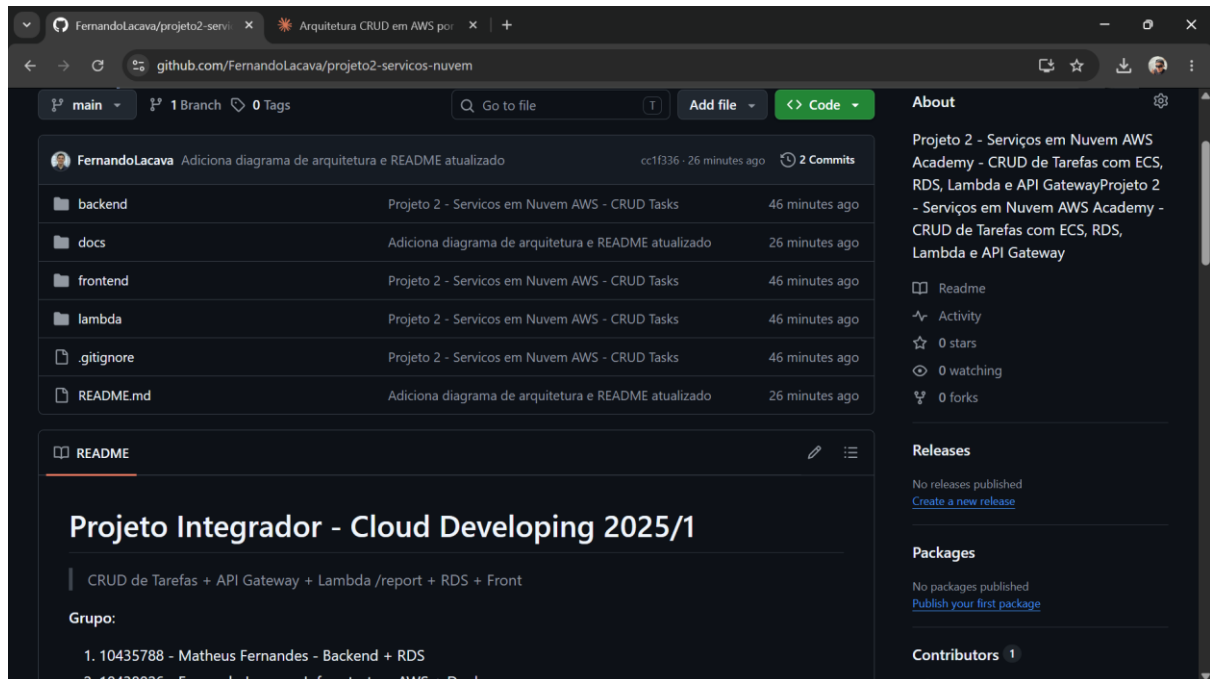
- Task Definition: tasks-frontend-task:1 (0.25 vCPU, 0.5 GB RAM)
- Service: tasks-frontend-service (1 tarefa em execucao)
- URL publica: http://35.153.51.11
- Comunicacao: exclusivamente via API Gateway (sem acesso direto ao backend)



8. Repositorio GitHub

O código-fonte completo do projeto está disponível no repositório público do GitHub.

- URL: <https://github.com/FernandoLacava/projeto2-servicos-nuvm>
- Branch principal: main



9. Conclusao

O projeto demonstrou com sucesso a implementacao de uma arquitetura distribuida na AWS, utilizando servicos gerenciados para cada camada da aplicacao. Os principais aprendizados foram:

- Configuracao de seguranca em camadas com Security Groups especificos por servico
- Isolamento do banco de dados em subnet privada sem acesso publico
- Containerizacao com Docker e orquestracao com ECS Fargate
- Uso de funcao serverless Lambda para processar relatorios sem acesso direto ao banco
- API Gateway como ponto central de entrada garantindo separacao de responsabilidades

Recursos Criados na AWS

Servico	Recurso
ECS Fargate	cloud-tasks-cluster (backend + frontend)
Amazon ECR	tasks-backend + tasks-frontend
Amazon RDS	tasks-db (PostgreSQL 16, db.t3.micro)
AWS Lambda	tasks-report (Node.js 22)
API Gateway	tasks-api (HTTP API)
VPC	Default VPC + 3 Security Groups